

C++ Concepts Active Issues List (Snapshot of Revision 4)

Document number: P0396R0

Reply to: Andrew Sutton <asutton@uakron.edu>

Date: 2016-06-24

Audience: CWG

Reference ISO/IEC TS 19217

This document contains the C++ core language issues for the Concepts Technical specification which the Committee (INCITS PL22.16 + WG21) has not yet acted, that is, issues with status [Ready](#), [Tentatively Ready](#), [Review](#), [Drafting](#), [Open](#), and [New](#). (See [Issue Status](#) below).

This document is part of a group of related documents that together describe the issues that have been raised regarding the C++ Standard. The other documents in the group are:

- [Table of Contents](#) for all concepts issues.
- [Index by Section](#) for all concepts issues.
- [Index by Status](#) for all concepts issues.
- [Closed Issues List](#)
- [Defect Reports List](#)

Section references in this document refer to the section numbering in document [WG21 N4549](#).

The purpose of these documents is to record the disposition of issues that have come before the Core Language Working Group of the ANSI (INCITS PL22.16) and ISO (WG21) C++ Standard Committee.

The issues in these lists are not necessarily formal ISO Defect Reports (DR's). While some issues will eventually be elevated to official Defect Report status, other issues will be disposed of in other ways.

For the most current official version of this document see <http://www.open-std.org/jtc1/sc22/wg21/>. Requests for further information about this document should include the document number above, reference ISO/IEC 19217, and be submitted to Information Technology Industry Council (ITI), 1250 Eye Street NW, Washington, DC 20005.

Information regarding C++ standardization can be found at <http://isocpp.org/std>.

Revision History

- R3: 2015-03-04 post-Jacksonville mailing
 - **Summary:**
 - 28 open issues, up by 1.
 - 7 closed issues, up by 1.
 - 35 issues total, up by 2.
 - **Details:**
 - Added the following 2 New issues: [34](#), [35](#).
 - Changed the following issue from Open to New: [33](#).

Issue Status

Issues progress through various statuses as the Core Language Working Group and, ultimately, the full PL22.16 and WG21 committees deliberate and act. For ease of reference, issues are grouped in these documents by their status. Issues have one of the following statuses:

Open: The issue is new or the working group has not yet formed an opinion on the issue. If a Suggested Resolution is given, it reflects the opinion of the issue's submitter, not necessarily that of the working group or the Committee as a whole.

Drafting: Informal consensus has been reached in the working group and is described in rough terms in a Tentative Resolution, although precise wording for the change is not yet available.

Review: Exact wording of a Proposed Resolution is now available for an issue on which the working group previously reached informal consensus.

Ready: The working group has reached consensus that a change in the working draft is required, the Proposed Resolution is correct, and the issue is ready to forward to the full Committee for ratification.

Tentatively Ready: Like "ready" except that the resolution was produced and approved by a subset of the working group membership between meetings. Persons not participating in these between-meeting activities are encouraged to review such resolutions carefully and to alert the working group with any problems that may be found.

DR: The full Committee has approved the item as a proposed defect report. The Proposed Resolution in an issue with this status reflects the best judgment of the Committee at this time regarding the action that will be taken to remedy the defect; however, the current wording of the Standard remains in effect until such time as a Technical Corrigendum or a revision of the Standard is issued by ISO.

Dup: The issue is identical to or a subset of another issue, identified in a Rationale statement.

NAD: The working group has reached consensus that the issue is not a defect in the Standard. A Rationale statement describes the working group's reasoning.

Extension: The working group has reached consensus that the issue is not a defect in the Standard but is a request for an extension to the language. The working group expresses no opinion on the merits of an issue with this status; however, the issue will be maintained on the list for possible future consideration as an extension proposal.

Active Issues

1. Relationship of implicit conversion constraints to `is_convertible`

Section: 14.10.1.1 [temp.constr.conv] **Status:** [Open](#) **Submitter:** CA **Opened:** 2015-05-18 **Last modified:** 2016-06-16

View all other [issues](#) in [temp.constr.conv].

View all issues with [Open](#) status.

Discussion:

More a suggestion than a comment: would a convertible-to-type example like the following be appropriate?

```
template <typename T> concept bool D =
requires (T a) {
    { a } -> int; // equivalent to std::is_convertible<T,int>::value ?
};
```

It could be here or in 14.10.1.6, but I get the feeling it would follow the `a==b` example nicely. There is something similar in the middle of the page, with concept C2, but it is more involved and contributes something else to reader comprehension.

During discussion in the July telecon, it was determined that there is a CWG issue related to this request. In particular, it is not obvious whether access checking was always applied or whether it depending on the context in which the constraint was evaluated. Unfortunately, the minutes did not capture which issue.

3. Allow `requires`-expressions in more contexts

Section: 5.1.4 [expr.prim.req] **Status:** [Open](#) **Submitter:** US **Opened:** 2015-05-18 **Last modified:** 2016-06-16

View other [active issues](#) in [expr.prim.req].

View all other [issues](#) in [expr.prim.req].

View all issues with [Open](#) status.

Discussion:

The following requirement seems overly restrictive, as it can be fairly easily (but tediously) be worked around: "A *requires-expression* shall appear only within a concept definition (7.1.7), or within the *requires-clause* of a template declaration (Clause 14) or function declaration (8.3.5)." (The tedious workaround for each concept C is to define an overload set consisting of two function templates, one unconstrained and returning false, the other constrained by C and returning true.)

Proposed change: Eliminate the requirement, thereby permitting other uses for this new kind of expression of type `bool`. (For

example, requires^Â-expressions might replace many or all of the Boolean type traits.) Additionally, in any context where a bool value is permitted, allow a concept[™]'s name plus suitable arguments to denote the truth value of the claim that "this combination of arguments satisfy this concept." (This syntax is currently valid in only certain contexts such as requires^Â-expressions.)

There is a paper addressing the *requires-expression* portion of this discussion. .

Issue [29](#) was created to specifically address the inability to evaluate concepts outside of SFINAE contexts.

6. Simplify concept definitions

Section: 7.1.7 [dcl.spec.concept] **Status:** [EWG](#) **Submitter:** US **Opened:** 2015-05-18 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [EWG](#) status.

Discussion:

Using concept as a *decl-specifier*, rather than forming a first class entity like a type or template, makes the feature appear more complex than it needs to be. Concepts would be simpler (for user and [we believe] the specification) if there was only one kind, rather than both function and variable syntax; the `bool` keyword would become redundant and the set of restrictions on concepts based on them being functions or variables would disappear. We will provide a paper in time for the Lenexa pre meeting mailing proposing a grammar that would give all concepts the form:

```
template <typename T>
concept C = predicate;
```

Also see issues [7](#) and [8](#) as they relate.

EWG is generally in favor of resolving this and related issues, but there is no consensus on how to proceed.

7. Remove function concepts

Section: 7.1.7 [dcl.spec.concept] **Status:** [EWG](#) **Submitter:** US **Opened:** 2015-05-18 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [EWG](#) status.

Discussion:

The syntactic distinction between a function concept and a variable concept seems to serve no useful purpose. A single concept syntax seems sufficient, and especially so once redundant elements are removed. Merge the two concept forms into one, streamlining the syntax by eliminating at least the following redundant elements: explicit `bool` (see comment below), explicit `return`, and the always empty parentheses constituting the function parameter list.

Also see issues [6](#) and [8](#) as they relate.

EWG is generally in favor of resolving this and related issues, but there is no consensus on how to proceed.

8. Implicit bool for concepts

Section: 7.1.7 [dcl.spec.concept] **Status:** [EWG](#) **Submitter:** US **Opened:** 2015-05-18 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [EWG](#) status.

Discussion:

Since a concept's type always must be `bool`, there seems little reason to require the source code to say so explicitly. Typing concept should be sufficient without also typing `bool` immediately afterward. Allow the compiler to supply `bool` (a) as the implicit return type for a function concept and (b) as the implicit type for a variable concept. (Note: this comment is implicitly accepted if issue [7](#) is accepted.)

Also see issues [6](#) and [7](#) as they relate.

EWG is generally in favor of resolving this and related issues, but there is no consensus on how to proceed.

11. Concerns about subsumption and equivalence rules

Section: 14.10.2 [temp.constr.decl] **Status:** [Open](#) **Submitter:** US **Opened:** 2015-05-18 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.decl].

View all other [issues](#) in [temp.constr.decl].

View all issues with [Open](#) status.

Discussion:

We have a broad concern that it is hard to understand the feature purely from the specification, especially the subsumption rules, and equivalence rules to know when two signatures declare the same function or are ambiguous equally constrained overloads, yet there is a lack of readily available implementations to test our understanding against. While the feature set of the TS looks good, we think one more iteration on the specification would be useful. Proposed solution: Recast the rules for subsumption as a mini grammar (distinct from the C++ grammar) as the English text appears to be trying to describe a grammar, but less formally, which leads to a potential lack of precision, and more confusion for the reader. We are not highlighting specific lack of precision at this time, as we have not emerged from confusion in time to file appropriate comments.

EWG believes this is strictly a wording issue. Returned to CWG.

13. Adjustment of parameter types in *requires-expressions*

Section: 5.1.4 [expr.prim.req] **Status:** [EWG](#) **Submitter:** Casey Carter **Opened:** 2015-05-28 **Last modified:** 2016-06-16

View other [active issues](#) in [expr.prim.req].

View all other [issues](#) in [expr.prim.req].

View all issues with [EWG](#) status.

Discussion:

It is unclear whether parameter types in a *requires-expression* are adjusted in the same way that parameter types for functions are adjusted.

Andrew Sutton says that it should be the case.

The TS editor has moved the issue back to EWG for discussion. If *requires-parameters* are adjusted in the same way that function parameters are adjusted, then we lose information within the concept; top-level cv-qualifiers are removed, types decay, etc.

Wording available:

Add the following sentence to 5.1.4/5:

The types of parameters declared in a *requires-expression* are adjusted according to the rules for forming function types in 8.3.5.

14. Concept checks on template template parameters are too restrictive

Section: 14.4.3 [temp.arg.template] **Status:** [EWG](#) **Submitter:** Roland Bock **Opened:** 2015-10-02 **Last modified:** 2016-06-16

View all issues with [EWG](#) status.

Discussion:

Consider a template that takes a `std::tuple` and copies its arguments into another template (sink), see attached code.

```
template<typename Tuple, template<typename...> class Sink>
using copy_tuple_args = ...
```

The `copy_tuple_args` template is generic, not caring about the nature of the copied arguments or the sink. This works fine as long as the sink is not constrained, e.g. if the sink is another tuple. But a constrained sink like this

```
template<Column... C>
struct column_list;
```

is not allowed according to the working paper since it is more constrained than the template parameter above. My `copy_tuple_arg` template will suddenly cease to work when I start to use concepts.

Wording available:

Modify [temp.arg.template]:

A *template-argument* (call it A) matches a template *template-parameter* (call it P) when each of the template parameters in the *template-parameter-list* of the *template-argument*'s corresponding class template or alias template (call it A) matches the corresponding template parameter in the *template-parameter-list* of P, and P is *either unconstrained or* at least as constrained as A according to the rules in 14.10.3.

15. Partial specialization of non-concept variable template as a concept definition

Section: 7.1.7 [dcl.spec.concept] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-10-02 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [Open](#) status.

Discussion:

Partial specialization of a concept definition is prohibited by 7.1.7 [dcl.spec.concept]p7; however, there appears to be no prohibition on a concept definition which is a partial specialization. e.g.,

```
template <typename T, typename U> bool C = true;
template <typename U> concept bool C<int, U> = false;
```

16. Concept and non-concept declarations of the same variable template

Section: 7.1.7 [dcl.spec.concept] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-07-09 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [Open](#) status.

Discussion:

It seems this is valid, and it is not clear if that is the intent.

```
namespace A {
    template <typename T> extern const bool C;
};
template <typename T> concept bool A::C = true;
```

Note: The C++14 restriction (removed in DR 1712) that the `constexpr` specifier be present on every declaration of a variable

template if any declaration has the `constexpr` specifier applies to the "physical" presence of the specifier.

Note: The first declaration of `A::C` is not a "variable concept". There is no restriction that a variable concept (i.e., "[a] variable template definition having the concept specifier") is the only declaration of the entity that it defines.

This seems to be a more general problem that being a "concept" is not a property of the entity, but of its definition.

Andrew Sutton: This should not be a valid definition.

17. Wording for subsumption

Section: 14.10.3 [temp.constr.order] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-05-25 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.order].

View all other [issues](#) in [temp.constr.order].

View all issues with [Open](#) status.

Discussion:

The wording in N4377 subclause 14.10.3 [temp.constr.order] bullet 2.2 reads:

- a disjunctive clause P_i subsumes a conjunctive clause Q_j if and only if each atomic constraint in P_i subsumes any atomic constraint Q_j , where
- ...

Firstly, as a conjunctive clause of a constraint in CNF, Q_j may be a disjunction as opposed to an atomic constraint. Adding "in" to give "any atomic constraint in Q_j " might be an editorial fix, but it does not fix the issue below.

Given "A and B" as P_0 , and "A" as Q_0 ; then my reading of the "each subsumes any" wording is that P_0 subsumes Q_0 if and only if each atomic constraint in P_0 (that is, A and B) subsumes A. That is: (A subsumes A) *and* (B subsumes A).

I believe the intent is that P_i subsumes Q_j if and only if there exists an atomic constraint, P_{ia} , in P_i for which there exists an atomic constraint, Q_{jb} , in Q_j such that P_{ia} subsumes Q_{jb} . I am not seeing a way to reconcile the current wording of bullet 2.2 with what I believe the intent is.

Also see issue [30](#). Addressing that issue will also close this one.

18. Predicate constraints that are not constant expressions

Section: 7.1.7 [dcl.spec.concept] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-05-26 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [Open](#) status.

Discussion:

The word "constant" only appears once in normative text in N4377. It is unclear to me whether a predicate constraint that is not a constant-expression is ill-formed (which would require a diagnostic for a non-templeid function with a requires-clause whose expression is not constant), or merely not satisfied.

It is useful to note that the "ill-formed" position leads to further issues where constant-expressions of the form $P \mid\mid Q$ may be written such that normalization of constraints will form $P \text{ or } Q$ where Q is a predicate constraint whose expression is not a constant-expression.

Faisal Vali, Andrew Sutton, and Gabriel Dos Reis agree that predicate constraints containing non-constant expressions should be ill-formed.

Andrew Sutton suggests: If Q is not dependent and not a constant expression, then the program would be ill-formed. Otherwise if, as a result of substituting during satisfaction, Q is not a constant expression the program is ill-formed.

However, if Q is dependent but never evaluated (because P is satisfied), the program is well-formed. This would be the same if substitution into Q would result in substitution failures (e.g., if Q is `X < T::value`).

19. Wording makes all constrained function definitions ill-formed

Section: 8.4.1 [dcl.fct.def.general] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-06-04 **Last modified:** 2016-06-16

View all issues with [Open](#) status.

Discussion:

In N4141 (C++14) subclause [dcl.fct.def.general] paragraph 2: The declarator in a function-definition shall have the form

D1 (parameter-declaration-clause) cv-qualifier-seq_{opt} ref-qualifier_{opt} exception-specification_{opt} attribute-specifier-seq_{opt} trailing-return-type_{opt}

as described in 8.3.5. A function shall be defined only in namespace or class scope.

The issue also occurs in N4141 subclause 12.1 [class.ctor] paragraph 1 and subclause 12.4 [class.dtor] paragraph 1.

20. Concept-names and overload sets with non-concept functions

Section: 7.1.7 [dcl.spec.concept] **Status:** [EWG](#) **Submitter:** Hubert Tong **Opened:** 2015-06-12 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [EWG](#) status.

Discussion:

An identifier is a concept-name if it refers to a set of concept definitions (7.1.7). Given the following, is C a concept-name?

```
template <typename T> concept bool C() { return true; }
template <typename T> int C(T t) { return 42; }

struct A { };

bool operator &&(A, int (*)(int));
bool operator &&(A, bool);

bool f(A a) { return a && C<int>; }
```

It seems that it is not a concept-name since it refers to a set of declarations which does not contain only concept definitions. If that is the intent, it should be made more clear.

21. Disambiguation rules for *requires*-clauses

Section: 8.3.5 [dcl.fct] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-06-08 **Last modified:** 2016-06-16

View all issues with [Open](#) status.

Discussion:

There is no disambiguation rule in C++14 PDS 19217 which requires a *type-specifier-seq* to consume as many *type-specifiers* as is available and avoid backtracking.

Given either

```
template <typename T> requires (bool)&T::operator short
unsigned int foo();
```

or

```
template <typename T> requires (bool)sizeof new (T::f()) short
unsigned int bar();
```

there is more than one successful parse and it is unclear whether the return type is `unsigned int` or `int`.

The after-the-function-declarator form of the *requires-clause* is also ambiguous:

```
struct X {};  
template<typename T> void f() requires (bool)sizeof new X  
{  
    // might be the function body, might be a brace-or-equal-init for X  
};
```

A 'max munch' rule would probably do the wrong thing for the above case, and likewise here:

```
template<typename T> requires (bool)sizeof new unsigned  
struct X { };
```

Proposed resolutions have been to:

- require parens
- specify rules to make this unambiguous
- specify the operand of a *requires-clause* as a new grammar.

No consensus has been reached.

22. Initializers are never *constraint-expressions*

Section: 7.1.7 [dcl.spec.concept] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-06-19 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [Open](#) status.

Discussion:

Initializers [dcl.init] are not expressions. However bullet 6.3 in [dcl.spec.concept] has a requirement where an "initializer shall be a constraint-expression".

Presumably the constraint is that the initializer shall have exactly one full-expression (that is,

```
template <typename T> concept bool C{};
```

is ill-formed), and that said full-expression is valid where a *constraint-expression* is required.

It also seems that C++14 subclause 8.5 [dcl.init] paragraph 2 should be removed or updated to exclude objects declared with the concept specifier in a manner similar to how it excludes objects declared with the `constexpr` specifier.

23. Associated constraints is a term defined for templates only

Section: 14.10.2 [temp.constr.decl] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-06-27 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.decl].

View all other [issues](#) in [temp.constr.decl].

View all issues with [Open](#) status.

Discussion:

The definition of "associated constraints" is in 14.10.2 [temp.constr.decl] paragraph 2 of N4377. Said definition only applies to templates, thus the "associated constraints" being referred to by N4377 subclause 1.3.1 [defns.signature] appears to be undefined.

24. Expression equivalence outside of declaration matching is novel

Section: 14.10.3 [temp.constr.order] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-06-27 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.order].

View all other [issues](#) in [temp.constr.order].

View all issues with [Open](#) status.

Discussion:

Consider the following case:

```
template <typename T> constexpr bool P = true;
constexpr bool Q = true;
```

```
template <typename T = int> requires P<T>
void foo(int = 0, T = 0);
```

```
template <typename U = int> requires P<U> && Q
void foo(U = 0, int = 0);
```

```
void bar() { foo(); }
```

It seems that `<T>` in the first declaration of `foo()` may be considered equivalent to `P<U>` in the second declaration of `foo()`; however, I would find it surprising if the call to `foo()` is unambiguous. Note that `T` and `U` are not involved in the partial ordering in this case aside from the determination of the more constrained template.

25. Block scope template declarations

Section: 14.10.3 [temp.constr.order] **Status:** [Open](#) **Submitter:** Hubert Tong **Opened:** 2015-06-03 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.order].

View all other [issues](#) in [temp.constr.order].

View all issues with [Open](#) status.

Discussion:

The following constraint in Clause 14 [temp] is insufficient to prevent the declaration of an abbreviated function template at block scope:

A template-declaration can appear only as a namespace scope or class scope declaration.

26. Function concepts not allowed to be declared in more than one TU

Section: 14.10.3 [temp.constr.order] **Status:** [Ready](#) **Submitter:** Hubert Tong **Opened:** 2015-06-09 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.order].

View all other [issues](#) in [temp.constr.order].

Discussion:

According to subclause 7.1.7 [dcl.spec.concept] paragraph 1:

When a function is declared to be a concept, it shall be the only declaration of that function.

Wording available:

Change that sentence to read:

When a function **template** is declared to be a concept, it shall be the only declaration of that function **template in the translation unit**.

27. Redundant restriction on function specifiers for concepts

Section: 7.1.7 [dcl.spec.concept] **Status:** [Open](#) **Submitter:** Nathan Wilson **Opened:** 2015-10-17 **Last modified:** 2016-06-16

View other [active issues](#) in [dcl.spec.concept].

View all other [issues](#) in [dcl.spec.concept].

View all issues with [Open](#) status.

Discussion:

According to subclause 7.1.7 [dcl.spec.concept] paragraph 5:

A function concept has the following restrictions: (5.1):

- No function-specifiers shall appear in its declaration (7.1.2).
- ...

Would that be redundant because of the restriction on function-specifiers being covered by subsection [dcl.spec.concept]p2 and the result of [dcl.spec.concept]p1, specifically, "The concept specifier shall be applied only to the definition of a function or variable template, declared in namespace scope"?

Wording available:

Strike the first bullet in [dcl.spec.concept]p5.

- **No function-specifiers shall appear in its declaration (7.1.2).**
- ...

28. Ordering of constraints involving fold expressions

Section: 14.10.3 [temp.constr.order] **Status:** [Open](#) **Submitter:** Robert Haberlach **Opened:** 2016-01-23 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.order].

View all other [issues](#) in [temp.constr.order].

View all issues with [Open](#) status.

Discussion:

Partial ordering by constraints doesn't regard fold expressions.

```
template <class T> concept bool A = std::is_move_constructible<T>::value;
template <class T> concept bool B = std::is_copy_constructible<T>::value;
template <class T> concept bool C = A<T> && B<T>;
```

```
template <class... _tx>
    requires (A<_tx> && ...)
void g(_tx... tx) {
    std::cout << "a\n";
}
```

```
template <class... _tx>
    requires (C<_tx> && ...)
void g(_tx... tx) {
    std::cout << "c\n";
}
```

Andrew Sutton: this is logically valid and seems like a reasonable extension. As a general rule, there may be many ways in which we can extend the constraint language to support these kinds of resolutions. Of course, this means that each such change potentially breaks changes.

This needs to apply to expansions of disjunctions also. This is done by inverting the subsumption (i.e., when Q subsumes P).

Wording available:

Augment 14.10.3 [temp.constr.order]p(2.3) thusly: an atomic constraint A subsumes another atomic constraint B if and only if either **the A and B are equivalent using the rules described in 14.10.1 to compare constraints, or A is of the form P && ... and B is of the form Q && ... or Q || ..., where P subsumes Q.**

29. Allow concepts to be evaluated in any context

Section: 14.10.2 [temp.constr.decl] **Status:** [Open](#) **Submitter:** Andrew Sutton **Opened:** 2016-02-27 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.decl].

View all other [issues](#) in [temp.constr.decl].

View all issues with [Open](#) status.

Discussion:

Currently, concepts are only guaranteed to be evaluable within the context of a requires-expression. Among other things, this guarantees that the following will not work: `static_assert(C<X>(), "");` // for some concrete X

Evaluating a concept in any context effectively requires that it be evaluated as if in a SFINAE context. The original design of concepts did not require this because it was not clear how easy this would be for all implementers. Based on EWG discussions in Kona, this appears to no longer be the case.

This issue is submitted as a result of an EWG straw poll taken during discussion of issue [3](#).

Wording available:

30. Normalization wording guarantees worst case performance for subsumption

Section: 14.10.2 [temp.constr.decl] **Status:** [Open](#) **Submitter:** Andrew Sutton **Opened:** 2016-02-27 **Last modified:** 2016-06-16

View other [active issues](#) in [temp.constr.decl].

View all other [issues](#) in [temp.constr.decl].

View all issues with [Open](#) status.

Discussion:

The current wording for normalization guarantees exponential performance during subsumption.

Addressing this issue first requires that we address issue [29](#). If concepts can be safely evaluated in any context, then we do not need to lift the entire constraint into the current instantiation.

The current phrasing also prohibits certain compiler optimizations by requiring a full expansion to atomic constraints. In particular, this does not admit early termination of the algorithm or memoization of comparisons.

Prior to the Skillman concepts meeting, there was a much more abstract wording for the subsumption algorithm. One resolution would be to revert the current wording to the older version.

Wording available:

31. Constrained-type-specifiers introduce non-type placeholders

Section: 7.1.6.4.2 [dcl.spec.auto.constr] **Status:** [Open](#) **Submitter:** Jason Merrill **Opened:** 2016-03-03 **Last modified:** 2016-06-19

View all other [issues](#) in [dcl.spec.auto.constr].

View all issues with [Open](#) status.

Discussion:

A *constrained-type-specifier* also denotes non-type constraints. The production name should reflect that.

33. Undefined behavior of *partial-concept-ids* in *template-introductions*

Section: 14.2 [temp.intro] **Status:** [New](#) **Submitter:** Hubert Tong **Opened:** 2016-05-04 **Last modified:** 2016-06-19

View all issues with [New](#) status.

Discussion:

There appear to be no semantics applied to the use of a partial-concept-id as the qualified-concept-name in a template-introduction. In particular, N4552 subclause 14.2 [temp.intro] refers to subclause 14.10.4 [temp.constr.resolve], which in turn does not specify what the concept argument list is in such a case.

Andrew Sutton: "disallowing this seems reasonable".

Wording available:

34. Premature substitution into associated constraints

Section: 14.6.4 [temp.friend] **Status:** [New](#) **Submitter:** Hubert Tong **Opened:** 2016-05-06 **Last modified:** 2016-06-19

View all issues with [New](#) status.

Discussion:

Given:

```
template <typename T>
    requires true || T::happy
void foo(T &&) { }

template <typename T>
struct A {
    friend void bar(A &&)
        requires true || T::happy {
    }
};

int main(void) {
    foo(0);
    bar(A<int>());
}
```

there appears to be wording (normative and otherwise), which would render the program ill-formed. I assume that is not the intent.

The wording involved is as follows: N4553 subclause 14.6.4 [temp.friend] paragraph 11: In the instantiation of such a class template (14.8), the template arguments are substituted into the constraints but not evaluated. [...] If substitution fails, the program is ill-formed.

N4553 subclause 14.9.2 [temp.deduct] paragraph 5: If the function template has associated constraints (14.10.2), the template arguments are substituted into the associated constraints without evaluating the resulting expression. If this substitution results in an invalid type or expression, type deduction fails.

Wording available:

35. Handwaving around instantiation of constraining elements

Section: 14.10.1 [temp.constr.constr] **Status:** [New](#) **Submitter:** Hubert Tong **Opened:** 2016-05-06 **Last modified:** 2016-06-19

View all issues with [New](#) status.

Discussion:

The TS often refers to substitution or instantiation of associated constraints; however, associated constraints are a derived property of what might be called "constraining elements", i.e., constrained-type-specifiers, constrained- parameters, template-introductions and constraint-expressions. It appears that the wording in N4553 subclause 14.10.1 [temp.constr.constr] is attempting to achieve "partial substitution" is trumped in many cases by substitution into constraining elements. For example, the associated constraints would evaluate false (or in a more realistic case, a check that `T::type` is a type) prior to considering `C<placeholder, typename T::type>()`, but it is unclear whether instantiating `A<int>` causes an error from the use of `T::type`:

```
template <typename T, typename U>
concept bool C() { return true; }
```

```
template <typename T>
struct A {
    template <typename U> requires false
    void foo(C<typename T::type>) { }
};

int main(void) { A<int> a; }
```

Wording available: